

Keychain

QA Staff Engineer

Take-Home Assignment

This assignment has two parts.

Part 1 is a take-home: build and submit a GitHub repo before your next interview. Part 2 happens live during the assignment review — we'll ask you to extend your framework on the spot. There is no separate brief for Part 2; what you build in Part 1 is what you'll extend.

The App

Your framework should be built against:

Conduit — a RealWorld example app

A Medium-style blogging platform — React frontend, Express REST API, SQLite. No external infrastructure required. The app covers: user registration and authentication, articles (create, edit, delete, list, filter), comments, user follows, and article favourites.

Repo: github.com/cirosantilli/node-express-sequelize-realworld-example-app

Getting the app running

```
git clone
https://github.com/cirosantilli/node-express-sequelize-realworld-example-app
cd node-express-sequelize-realworld-example-app
npm install && npm start
# UI: http://localhost:4101    API: http://localhost:3000/api
```

The app ships with its own tests. Ignore them — they are not the framework we're evaluating. Your test suite lives in a separate, new repo.

Part 1 — The Framework (take-home)

Build an agentic-first test framework for the app above. Submit it as a public GitHub repo.

By 'agentic-first' we mean: the framework is designed from the ground up to work correctly when an AI coding agent — Claude Code, Copilot, or similar — contributes a test, factory, or page object. Not just documented. Designed. What that means in practice is your call — we want to see how you think about it.

What the repo must have

- Working tests — at least 3–5 that cover critical flows. At least one API test and one UI test. Tests should run against the app when it's started locally.
- Agentic infrastructure — whatever you believe makes a test framework genuinely agent-native. This is intentionally open. Show your thinking through what you build, not just through documentation.
- DECISIONS.md at the repo root — 3–5 key architecture decisions. For each: what you chose, what you rejected and why, and what constraint would flip the decision.
- AI_USAGE.md at the repo root — how you actually used AI tools during this assignment. Specific prompts, specific responses, specific corrections. At least one example where the AI produced something wrong and you caught it.

Scope & Out-of-Scope

In scope	Out of scope
• Playwright + TypeScript as the test framework • API and UI test coverage (both required) • Shared auth and data management patterns • Agentic infrastructure (you define what this looks like) • Locally runnable — no deployment required	• CI/CD pipeline (mention it in DECISIONS.md if you'd add it) • Full test coverage — depth over breadth • Performance or load testing • Multi-browser support • Modifying the app under test

Scope tip:

If something is genuinely ambiguous, make a decision and document it in DECISIONS.md. We're more interested in your reasoning than your answer. Don't ask for permission to make architectural choices — that's the job.

Submission

- A public GitHub repo link, submitted at least 24 hours before your assignment review interview
- DECISIONS.md and AI_USAGE.md at the repo root
- A README that lets a new engineer clone, install, and run the tests without asking you anything